robust unlimited sampling

code for all the plots and recovery algorithm for modulo operator can be found here: Click here to open Colab notebook

Krunal Vaghela

May 2025

1 problem formulation

similar to that in sampling.pdf

2 our aim

we propose a generalized algorithm to reconstruct the input signal regardless of kind of operator used.

3 Use of special attenuators

So the idea behind using attenuators is to attenuate the part of the signal that exceeds the dynamic range by some gain and again multiply by the gain to reconstruct the signal.So we must know how much gain to apply and specifically where to apply the gain.AGC use a closed-loop feedback system to identify where to apply this gain and how much to apply.Companding also works on similar principles but it also magnifies smaller amplitudes so that they dont go below noise floor.

4 main idea about our recovery algorithm

Our algorithm uses the fact that the residual signal, the difference between the true signal and the output of the nonlinear operator, is time-limited for bandlimited input signals.We also assume our sampling period is small enough so that there also exist one or more samples such that $|f(nT_s)| < \lambda$.The assumption ensures that there are folded samples on which unfolding methods can be applied.
we assume some integral multiple of 2λ needs to be added in order to get original signal from modulo samples.We use the properties of finite energy bandlimited signals to construct the algo:

- Time domain seperation:in text they are using some lemma which is new to me called Riemann-Llebesgue lemma:which says Fourier transform or Laplace transform of an L1 function vanishes at infinity. It is of importance in harmonic analysis and asymptotic analysis.So L1 functions are basically functions which are absolutely integrable.
  Now if our signal is band limited then obviously from my intuition and experience of all functions i have seen their fourier transform is absolutely integrable..like for example fourier transform(sinc(t))= box function which is finitely integrable,fourier transform(constant function)=impulse..which is also absolutely integrable.So if Fourier tranform of function(bandlimited) is L1

function then using the lemma we can say fourier transform(fourier transform(bandlimited function))=original function.. will tend to zero at infinity.I know this is very vague way to explain things but I have used the property of duality here when I say fourier transform(fourier transform(bandlimited function))=original function. So we use the fact that our function is decaying after some point and the input function is in the dynamic range .The "n" after which we see this decaying property is what we call N_λ.And after $n > N_\lambda$ we say z=0 and hence z becomes time limited function.**There is saying that a there exist no function which is time limited as well as band limited.This gives me some vibes of "Heisenberg uncertainty principle"...trade between spread of time and frequency domains...but also note that there do exist functions which are time unlimited and band unlimited..its crazy how many things in nature gave us some signs**.

- Fourier domain separation:since f is band limited ,therefore after ω_m output signal is equal to z.

$$\mathcal{F}_\lambda\left(e^{j\omega T_s}\right) = \mathcal{Z}(e^{j\omega T_s}) \quad \text{for } \omega_m < |\omega| < \frac{\omega_s}{2}$$

$$\text{using the fact that } z[n] = 0 \quad \text{for } |n| > N_\lambda$$

$$\Rightarrow \mathcal{Z}(e^{j\omega T_s}) = \sum_{n=-N_\lambda}^{N_\lambda} z(nT_s)e^{-jnT_s\omega}$$

$$= \mathcal{F}_\lambda(e^{j\omega T_s})$$

So we need to find value of z at every n
$\Rightarrow$ we have n variables, we need n equations.

To get n equations we need n ω's.
i.e., we sample $\mathcal{F}_\lambda(e^{j\omega T_s})$ at n points
between $\omega_m < |\omega| < \frac{\omega_s}{2}$

$$\mathcal{F}_\lambda(e^{j\omega_1 T_s}) = \sum_{n=-N_\lambda}^{N_\lambda} z(nT_s)e^{-jnT_s\omega_1}$$

$$\vdots \qquad\qquad \vdots$$

$$\mathcal{F}_\lambda(e^{j\omega_n T_s}) = \sum_{n=-N_\lambda}^{N_\lambda} z(nT_s)e^{-jnT_s\omega_n}$$

Using matrix representation:

$$\begin{bmatrix} \mathcal{F}_\lambda(e^{j\omega_1 T_s}) \\ \mathcal{F}_\lambda(e^{j\omega_2 T_s}) \\ \vdots \\ \mathcal{F}_\lambda(e^{j\omega_n T_s}) \end{bmatrix} = \begin{bmatrix} e^{-j(-N_\lambda)T_s\omega_1} & \dots & e^{-jnT_s\omega_1} & \dots & e^{-j(N_\lambda)T_s\omega_1} \\ e^{-j(-N_\lambda)T_s\omega_2} & \dots & e^{-jnT_s\omega_2} & \dots & e^{-j(N_\lambda)T_s\omega_2} \\ \vdots & & \vdots & & \vdots \\ e^{-j(-N_\lambda)T_s\omega_n} & \dots & e^{-jnT_s\omega_n} & \dots & e^{-j(N_\lambda)T_s\omega_n} \end{bmatrix} \begin{bmatrix} z(-N_\lambda T_s) \\ z((-N_\lambda+1)T_s) \\ \vdots \\ z(0) \\ \vdots \\ z((N_\lambda-1)T_s) \\ z(N_\lambda T_s) \end{bmatrix}$$

Let:

$$\mathcal{F}_\lambda = VZ$$

Where:

$$V \Rightarrow (2N_\lambda + 1) \times (2N_\lambda + 1) \quad \text{(Vandermonde matrix)}$$

5 limitations of inversion of matrix hence we now use PSD

Projected Gradient Descent via Ball-Rolling Analogy

We can interpret the iterative algorithm for solving the optimization problem in $\mathrm{B}^2\mathrm{R}^2$ as a physical process: a ball rolling on a bumpy terrain, constrained to remain within a fenced region.

At each iteration:

1. The ball (current estimate $\mathbf{z}^{k-1}$) is rolled downhill following the negative gradient of the cost function $C(\mathbf{z})$:
$$\mathbf{y}^k = \mathbf{z}^{k-1} - \gamma_k \nabla C(\mathbf{z}^{k-1}),$$
where γ_k is the step size and $\nabla C(\mathbf{z}) = \mathcal{F}_\rho^* \mathcal{F}_\rho(\mathbf{z} - \mathbf{f}_\lambda)$.
2. After moving, the ball may leave the valid signal space $\mathcal{S}_{N_\lambda}$. So we project it back into the valid space:
$$\mathbf{z}^k = \mathcal{P}_{\mathcal{S}_{N_\lambda}}(\mathbf{y}^k).$$

This process is repeated until the ball settles in a low point *inside the fence*, giving the recovered signal.

Symbol	Meaning	Ball Analogy
$\mathbf{z}^k$	Signal estimate at step k	Ball position
$\nabla C(\mathbf{z})$	Gradient of cost	Slope of the terrain
γ_k	Step size	Push strength on the ball
$\mathcal{S}_{N_\lambda}$	Time-limited signal space	Fenced valid region
$\mathcal{P}_{\mathcal{S}_{N_\lambda}}$	Projection operator	Picking up the ball and dropping it inside the fence
$\mathcal{F}_\rho^* \mathcal{F}_\rho$	Highpass operator	Shape of terrain guiding the descent

Table 1: Ball rolling analogy for projected gradient descent in $\mathrm{B}^2\mathrm{R}^2$.

6 implementing the algo

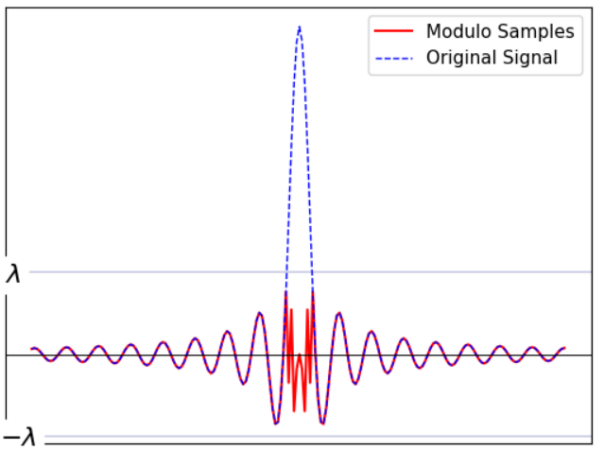

Figure 1: modulated signal-only one sinc wave and Lambda = 0.25

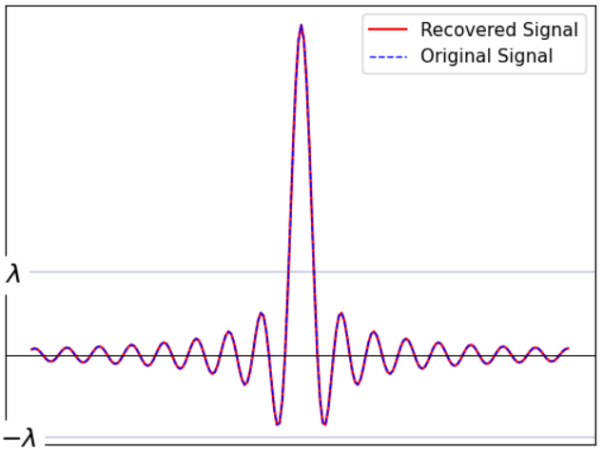

Figure 2: recovered signal with tuned parameter: N$_l ambda = 4$

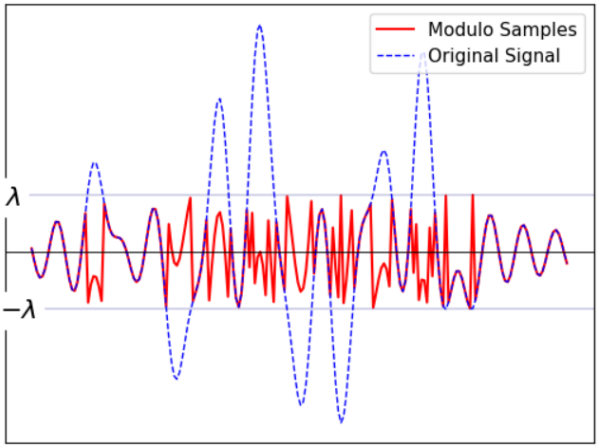

Figure 3: modulated signal-sum of 10 sinc wave and Lambda = 0.25

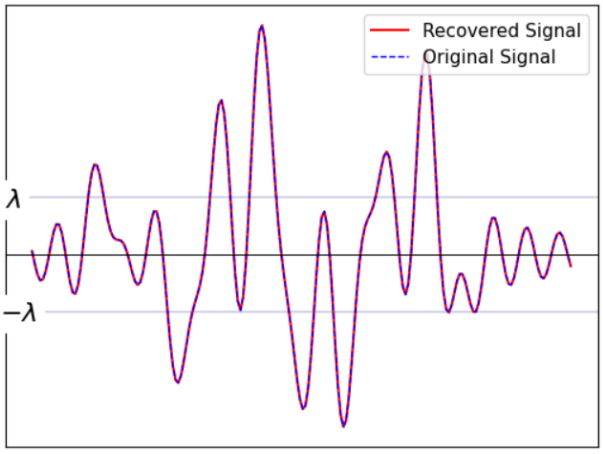

Figure 4: recovered signal with tuned parameter: N$_l ambda = 100$

Final Remarks

Throughout this report, I have tried my best to present my understanding of the equations and concepts involved. I have focused not only on writing down the mathematical formulations but

also on extracting intuitive meaning from them. In particular, I enjoy relating abstract concepts to physical analogies or familiar ideas (as seen, for example, in the ball-rolling perspective on projected gradient descent).

If any of the intuitions or interpretations I have offered are incorrect or sounded unintentionally amusing, I would be genuinely glad to be corrected or receive feedback, as this would only deepen and enhance my understanding.

Additionally, I would like to acknowledge that while the implementation of the algorithm was carried out by me, I leveraged the help of AI tools and online resources to understand, debug, and structure the code. Therefore, the core logic and prompts were my own, but not all parts of the code were written from scratch manually.